

Lifelong Opportunity Guide 89 — Software Developer

Version: 2.0 controlled working master

Language: English

Primary U.S. benchmark: O*NET-SOC 15-1252.00 — Software Developers

Canada comparison: NOC 21232 — Software developers and programmers

Colombia comparison: CUOC 25120 — Desarrolladores de software

Review date: 2026-08-22

What this career is

A Software Developer analyzes needs, designs and builds software, tests and improves it, documents decisions, and supports software through change and operation. Depending on the role, the product may be a desktop application, mobile app, cloud service, embedded component, internal business system, API, data-processing service, platform component or specialized utility.

The title is broad. Some developers focus on application features. Others work on distributed systems, cloud platforms, DevOps tooling, embedded software, enterprise systems, data services or infrastructure software. The exact programming language matters less over a career than the ability to understand requirements, design maintainable solutions, test assumptions, debug failures, protect data, work safely in teams and learn new technologies.

The United States has a direct benchmark in **O*NET-SOC 15-1252.00 — Software Developers**, Bright Outlook and updated in 2026. Canada maps directly to **NOC 21232 — Software developers and programmers**. Colombia has a direct comparison in **CUOC 25120 — Desarrolladores de software**.

Why software development remains a strong field

Software supports finance, logistics, healthcare, government, communications, transportation, manufacturing, cybersecurity, media, education and nearly every digital business process.

Current U.S. O*NET/BLS evidence projects **16% employment growth from 2024 to 2034** with about **115,200 annual openings**, including growth and replacement. Those numbers describe the occupation nationally; they do not guarantee a job for an individual applicant.

A durable developer combines:

- problem definition;
- programming fundamentals;
- data structures and algorithms;
- software design;
- APIs and data persistence;
- testing;
- debugging and observability;
- security and privacy awareness;
- version control and collaboration;
- documentation;
- release discipline;
- responsible use of automation and AI.

Common role families

Application developer

Builds user-facing or business applications and may work across interface, service and data layers.

Back-end/service developer

Focuses on APIs, business logic, data persistence, messaging, caching, integration and service reliability.

Platform/infrastructure developer

Builds tooling, internal platforms, automation, deployment systems and infrastructure-facing software.

Mobile developer

Builds native or cross-platform mobile applications and handles device lifecycle, permissions, storage, networking and app-store requirements.

Embedded/systems developer

Works closer to operating systems, devices, hardware, networking or performance-sensitive code.

DevOps/SRE-adjacent developer

May build deployment, observability, infrastructure automation and reliability tooling. DevOps and SRE are distinct disciplines, but software-development skills overlap.

Start with requirements

Strong development begins with a problem and an agreed outcome, not with code.

Clarify:

1. What user or business problem are we solving?
2. What behavior is required?
3. What is explicitly out of scope?
4. What inputs and outputs exist?
5. What performance, availability, privacy or security requirements apply?
6. Which interfaces and systems are affected?
7. What evidence will show that the change works?
8. Who approves the requirement and release?

If requirements conflict or are unclear, record the ambiguity and escalate. Do not silently invent product behavior after implementation.

Design before implementation

Design can be lightweight or formal depending on risk. Useful questions include:

- Which component owns the behavior?
- What data must be stored?
- What API or interface contract applies?
- What happens when dependencies fail?
- What must be backward compatible?
- What is the rollback or migration path?
- What access controls are needed?
- How will the system be observed?
- How will the change be tested?

Architecture decisions may belong to senior engineers, architects or platform/security teams. A developer should contribute evidence and tradeoffs without claiming authority they do not have.

Programming fundamentals

Useful foundations across languages include:

- variables and types;
- expressions and control flow;
- functions/methods;
- modules/packages;

- collections;
- exceptions/errors;
- input/output;
- interfaces/abstractions;
- object-oriented and functional concepts where relevant;
- asynchronous/concurrent behavior;
- memory/resource management at the level required by the stack;
- testing and debugging.

Do not chase every language. Learn one stack deeply enough to understand how programs actually execute, then transfer the concepts.

Data structures and algorithms

Developers should understand practical tradeoffs involving:

- arrays/lists;
- maps/dictionaries;
- sets;
- stacks/queues;
- trees/graphs at a conceptual level;
- sorting/searching;
- iteration;
- time and space complexity;
- choosing a structure that fits the workload.

Not every job requires advanced algorithm interviews, but understanding complexity helps prevent inefficient designs and makes debugging performance easier.

Current technology signals

O*NET employer-posting data for 2025 show strong demand for:

- Python **29%**;
- AWS **26%**;
- Java **25%**;
- SQL **24%**;
- JavaScript **20%**;
- Azure **19%**;
- Kubernetes **14%**;
- Git **14%**;
- RESTful API **13%**;
- React **13%**;
- Docker **13%**;

- C# **12%**;
- C++ **10%**;
- Angular **10%**;
- CSS **9%**;
- Linux **9%**;
- Jenkins CI **8%**;
- HTML **8%**;
- TypeScript **8%**;
- Node.js, JIRA, GitHub and NoSQL **7%**;
- PostgreSQL, Terraform, Kafka and C **6%**;
- Spring Boot, Go and Spring Framework **5%**.

These percentages are market signals, not universal requirements. The right stack depends on product, employer and specialization.

APIs and contracts

Interfaces should have defined behavior. Developers commonly work with:

- REST/HTTP APIs;
- RPC or messaging interfaces;
- event streams;
- SDK/library contracts;
- database schemas;
- file formats.

Document inputs, outputs, errors, versioning expectations and compatibility. Avoid changing public/consumer-facing behavior silently.

Databases and persistence

Useful concepts include:

- tables/documents;
- keys and relationships;
- indexes;
- transactions;
- consistency;
- migrations;
- connection management;
- caching;
- retention;
- backup/recovery ownership.

Use parameterized database access or approved ORM/query mechanisms. Do not concatenate untrusted input into queries.

Authentication and authorization

Authentication establishes identity. Authorization determines permitted actions.

Developers should follow the application's approved identity model, least-privilege principles and server-side authorization rules. Hiding a UI control does not protect a server resource. Do not weaken authorization simply to make a feature pass a test.

Source control and code review

Git or another version-control system provides change history and collaboration.

Useful practices include:

- focused commits;
- meaningful messages;
- branches according to team policy;
- pull/merge requests;
- peer review;
- conflict resolution;
- tags/releases where used.

Never commit credentials, API keys, tokens, certificates or private secrets. If a secret enters repository history, follow the organization's rotation/incident process rather than merely deleting the visible line.

Testing and QA

Developers may use:

- unit tests;
- integration tests;
- contract/API tests;
- component tests;
- end-to-end tests;
- regression tests;
- performance tests;
- accessibility checks;
- security tests within authorization.

A test should have meaningful assertions and controlled data. Passing tests are evidence, not proof that software is bug-free or secure.

Work constructively with QA. A reproducible defect report is evidence to investigate, not a personal criticism.

Debugging

A disciplined debugging workflow can be:

1. reproduce the problem;
2. identify the affected version/environment;
3. reduce the problem to the smallest useful case;
4. inspect logs, metrics, traces, inputs and state;
5. form a hypothesis;
6. test the hypothesis;
7. implement the smallest safe fix;
8. add/adjust tests;
9. retest and monitor.

Avoid random production changes when evidence is weak.

Observability

Software should provide enough evidence to understand behavior in operation.

Relevant tools and practices include:

- structured logs;
- metrics;
- distributed tracing where appropriate;
- request/correlation identifiers;
- health checks;
- dashboards and alerts;
- incident records.

Do not log secrets, authentication tokens, full payment details or unnecessary personal information.

Error handling and resilience

Plan for failure:

- invalid input;
- dependency timeouts;
- partial failures;
- unavailable services;
- duplicate events;
- retries;

- degraded performance;
- resource exhaustion.

Use bounded retries/backoff and idempotency where applicable. Do not retry every failure blindly; retries can amplify outages.

Concurrency and asynchronous work

Some systems process multiple operations at once. Developers may need to understand:

- race conditions;
- synchronization;
- shared state;
- queues;
- ordering;
- eventual consistency;
- cancellation/timeouts;
- duplicate processing.

The depth required varies by role, but asynchronous code should not be treated as automatically safe just because it compiles.

Dependencies and software supply chain

Third-party libraries and packages create productivity and risk.

Good practice can include:

- use only needed dependencies;
- prefer maintained, trustworthy sources;
- review security advisories;
- verify versions/checksums through approved tooling;
- review licenses where required;
- update supported packages;
- test upgrades;
- avoid copy-pasting unknown code into production.

Configuration and secrets

Keep environment-specific configuration and secrets in approved systems. Protect:

- database credentials;
- API keys;
- cloud credentials;
- OAuth client secrets;

- signing keys;
- certificates;
- encryption keys.

A local configuration file is not automatically safe. Follow repository ignore rules and secrets-management policy.

CI/CD and build pipelines

A software pipeline can compile/build, test, scan, package and deploy software.

A trustworthy pipeline should identify:

- source commit/version;
- dependencies;
- test results;
- artifacts;
- environment;
- approvals/gates;
- release status.

Do not bypass required pipeline gates because a manual deployment is faster.

Deployment, change and rollback

Before production change, understand:

- release version;
- configuration;
- migrations;
- dependencies;
- monitoring;
- rollback/forward-fix plan;
- data compatibility;
- responsible approver.

Technical access is not the same as release authority.

Cloud and shared responsibility

AWS, Azure, containers and Kubernetes appear frequently in current postings. Cloud platforms automate infrastructure but do not automatically own every responsibility for identity, application code, configuration, secrets, data and access.

Follow the provider's service-specific shared-responsibility model and organizational architecture.

Performance and capacity

Performance work may involve:

- CPU/memory;
- I/O;
- network latency;
- database queries;
- caching;
- concurrency;
- connection pools;
- payload size;
- service dependencies;
- load testing within authorized limits.

Measure before optimizing. Do not assume the most complex solution is the fastest.

Secure development

Security should be considered throughout the lifecycle. Relevant controls can include:

- input validation;
- safe output handling;
- authentication/authorization;
- parameterized queries;
- secure defaults;
- secrets management;
- dependency management;
- encryption through approved libraries/services;
- logging/monitoring;
- code review;
- security testing within authorization.

NIST SSDF and OWASP resources are useful learning references. They do not grant permission to perform intrusive security testing.

Privacy and data minimization

Use only data needed for approved purposes. Follow organizational rules for:

- access;

- collection;
- retention/deletion;
- test data;
- exports;
- logging;
- analytics;
- incident reporting.

Do not invent legal requirements. Escalate privacy/legal questions to responsible roles.

Accessibility and inclusive software

Depending on product, developers can support accessibility through semantic structures, keyboard operation, labels, focus management, contrast, understandable errors, zoom/reflow, assistive-technology compatibility and accessible documents/interfaces.

Automated scanners detect only part of accessibility problems. A passing scan does not prove legal compliance.

Responsible AI in software development

AI can assist with code explanation, scaffolding, refactoring, tests, synthetic data, documentation and debugging when policy allows.

Human accountability remains essential.

Do not:

- upload proprietary source code, customer data, secrets, credentials or unreleased information to unapproved AI tools;
- assume generated libraries/APIs exist;
- merge code you do not understand;
- accept insecure patterns because code compiles;
- skip tests/review;
- ignore license/dependency implications;
- allow autonomous production deployment outside governance;
- treat generated explanations as execution evidence.

Documentation and maintainability

Good software can be maintained by someone other than its original author.

Document, where useful:

- purpose;
- architecture decisions;

- setup;
- configuration;
- APIs/interfaces;
- data models;
- deployment;
- runbooks;
- limitations;
- troubleshooting;
- ownership.

Prefer clear code and small, reviewed changes over unnecessary cleverness.

Ethical and professional boundaries

A Software Developer should not:

- fabricate test or performance results;
- hide known high-impact defects;
- deploy without assigned authority;
- bypass reviews/security controls to meet a deadline;
- commit secrets;
- use production data in personal demos;
- weaken authorization without approved requirements;
- perform security exploitation outside explicit permission;
- claim a system is secure/bug-free because tests passed;
- publish employer source code/private architecture;
- present legal, clinical, accounting or safety conclusions outside assigned competence.

Education and entry pathways — United States

O*NET places Software Developers in **Job Zone Four — Considerable Preparation Needed**. Current new-hire education responses are approximately 85% bachelor's, 5% associate and 5% master's degree. These describe the occupation, not every individual job posting.

CareerOneStop/American Job Centers can help locate WIOA-approved and other training. Eligibility and funding vary locally.

O*NET lists approved apprenticeship titles including **Application Developer**, **Commercial Drone Software Developer**, **Devops Engineer (Nof)** and **Software Developer (Nof)**. Verify live openings through [Apprenticeship.gov](https://www.apprenticeship.gov).

Canada

Canada Job Bank maps Software Developer to **NOC 21232 — Software developers and programmers**.

Typical current requirements include a bachelor's degree in computer science/software engineering or another program with significant programming, **or** a college program in computer science/related field. Job Bank currently identifies the occupation as **not regulated in Canada**.

Canada wages

- **C\$30.00/hour low;**
- **C\$48.08/hour median;**
- **C\$76.92/hour high.**

Canada outlook

National 2024–2033 labour demand and supply are expected to be broadly in line. Three-year provincial prospects vary; verify the location-specific outlook.

Colombia

CUOC 25120 — Desarrolladores de software is a direct match at competency level 4. It covers analysis, design, development, testing, maintenance and implementation of software/ICT solutions.

The guide does not manufacture a representative current Colombian national Software Developer salary from historical/nonrepresentative profile ranges.

SENA pathway

Análisis y desarrollo de software

- Tecnólogo;
- **3,984 hours;**
- titulada training;
- requirements, analysis, design, development, implementation and software-quality competencies;
- live cohort, modality, seats and admission must be verified.

Latin America and Caribbean

ILO/Cinterfor can help locate national vocational-training institutions. It does not guarantee a current software-development course, scholarship, seat or funding award.

Current wages and outlook

United States official data

Percentile	Annual	Hourly
10th	\$82,460	\$39.64
25th	\$105,210	\$50.58
Median	\$135,980	\$65.38
75th	\$171,980	\$82.68
90th	\$214,670	\$103.21

2024-2034:

- employment 2024: **1,693,800**;
- projected employment 2034: **1,961,400**;
- growth: **16%**;
- annual openings: **115,200**.

Current adjacent non-government context

The Indeed Software Developer URL currently redirects to a **Software Engineer** salary page. That page, updated **August 10, 2026**, reports approximately:

- average **\$135,356/year**;
- low **\$80,008/year**;
- high **\$228,992/year**;
- **39.3k** observations from job postings over **36 months**;
- **\$5,000/year** cash-bonus context.

Because the live page is labelled Software Engineer, these figures are adjacent market context only—not an exact title-specific Software Developer statistic and not an official wage source.

Practical learning sequence

Stage 1 — programming foundations

Learn one language, Git, debugging, tests, basic data structures and simple command-line tooling.

Stage 2 — applications and data

Build a small application with APIs/interfaces, persistence, validation, error handling and tests.

Stage 3 — engineering practices

Add code review, CI, structured logging, configuration/secrets and dependency management.

Stage 4 — production thinking

Learn deployment, rollback, observability, cloud basics, security and performance.

Stage 5 — specialization

Develop deeper skill in application development, back end, mobile, platform/infrastructure, cloud, embedded systems, data services or another domain.

Safe portfolio projects

Use self-built, open-source, licensed or demo software and synthetic/public data.

A useful portfolio can demonstrate:

1. problem/requirements;
2. design decisions;
3. clean source history;
4. API/data work;
5. automated tests;
6. secure configuration;
7. CI build;
8. logs/error handling;
9. README/documentation;
10. controlled demo deployment;
11. limitations and future work.

Do not publish employer code, customer data, credentials, internal architecture/endpoints or unauthorized vulnerabilities.

Four-week starter plan

Week 1

Choose one language and build small command-line or local programs; use Git and unit tests.

Week 2

Build a small application/service with an API or interface and persistent data; add validation and error handling.

Week 3

Add integration tests, structured logging, environment configuration and a simple CI pipeline.

Week 4

Document architecture, security assumptions and limitations; create a controlled demo release and write accurate résumé bullets.

Job-search titles

- Software Developer;
- Junior Software Developer;
- Software Engineer;
- Application Developer;
- Back-End Developer;
- Platform Developer;
- Cloud Developer;
- Systems Developer;
- Integration Developer;
- Java Developer;
- Python Developer;
- .NET Developer;
- Mobile Developer;
- DevOps Engineer (where development is central).

Read actual duties carefully. Similar titles can have very different engineering depth and production responsibility.

Questions before accepting a role

- What type of software does the team own?
- Which languages/frameworks are actually used?
- How are requirements and architecture decisions made?
- How are code reviews handled?
- What automated testing is expected?
- How are releases approved and rolled back?
- Who handles production incidents/on-call?
- How are secrets and dependencies managed?
- What security/privacy responsibilities belong to developers?
- How is technical debt prioritized?
- What distinguishes junior from senior performance?
- Does the employer fund training or certifications?

Sources and verification links

United States

- O*NET details: [Source link](#)
- O*NET summary: [Source link](#)
- O*NET Job Zone: [Source link](#)
- O*NET wages: [Source link](#)
- O*NET outlook: [Source link](#)
- O*NET technologies: [Source link](#)
- CareerOneStop WIOA: [Source link](#)
- Apprenticeship.gov: <https://www.apprenticeship.gov/>
- Indeed adjacent market context: [Source link](#)

Canada

- Job Bank summary: [Source link](#)
- Job Bank requirements: [Source link](#)
- Job Bank wages: [Source link](#)
- Job Bank outlook: [Source link](#)
- Canada training: [Source link](#)

Colombia and Latin America

- OCUPACOL CUOC 25120: [Source link](#)
- SENA Análisis y desarrollo de software: [Source link](#)
- OIT/Cinterfor: <https://www.oitcinterfor.org/statsfp/paises>

Security, AI and accessibility

- NIST SSDF: <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP WSTG: [Source link](#)
- NIST AI RMF: [Source link](#)
- Section 508: <https://www.section508.gov/create/>
- WCAG 2.2: <https://www.w3.org/TR/WCAG22/>

Important notice

This guide provides general education and career-planning information. It does not guarantee employment, income, admission, funding, apprenticeship placement, certification, promotion, security, accessibility compliance or any other outcome.

No independent human certification, professional accreditation, legal review, security assessment, accessibility certification, cloud/vendor certification or certified translation is claimed unless separately documented.

Author and AI assistance

Created and directed by **Alberto “Al” Leiva**. ChatGPT supported research, organization, editing, translation support and document preparation under the author's direction. The author remains responsible for editorial and publication decisions.

License

Unless a file states otherwise, this material is licensed under **CC BY-NC-SA 4.0**.